Authenticate the Reads, Not the Code: Isolating Untrusted Database Executors with Capsule Transactions

Ian Lucas Beé

Preprint draft, July 2026. Figures 1–4 are typeset as annotated ASCII/pseudocode diagrams. Companion artifacts: the technical report "The Capsule Transaction Theorem" (27 pp., July 2026), its Seal-v3 and Session Protocol Addendum, and the Aster repository. Claim-by-claim evidence status is tracked in `paper/CLAIMS.md`*. This revision reconciles the text with the v0.8 tree (2026-07-28): statements about shipped mechanisms refer to v0.8; every measurement is from the dated v0.6/v0.7 campaigns and remains labeled as such.*

Abstract

Platforms increasingly run code they do not trust (AI-generated application logic, third-party plugins, multi-tenant functions) next to a database they do. Our threat model makes that distrust total: the executor is Byzantine from its first instruction, arbitrary probabilistic-polynomial-time code holding no database credential and, under the A12 deployment profile the model assumes, which v0.8 ships as a hardened container profile (rootless, read-only, network-less cells; not independently audited), no network or filesystem, whose only channel is a Unix-domain socket to a trusted broker that owns storage, the MAC key, and a single-writer lease. The prevailing defenses isolate the *compute* (microVMs, enclaves) and broker the *outbound credential*, but the code still holds some path to data; as recent incidents show, isolation is one SSRF away from credential exfiltration. We take the opposite cut: the executor never touches storage at all. The broker serves each untrusted invocation cryptographically **sealed, per-invocation data capsules**; at commit, it validates the executor's **MAC-authenticated read-set** with classical optimistic backward validation before appending. We prove the *Capsule Transaction Theorem*: committed mutations are strictly serializable over the declared, authenticated read/write sets (snapshot reads may be stale), and a compromised executor is exactly as powerful as a malicious-but-authorized application: it cannot fabricate a read, forge a conflict, or append through a stale lease epoch. A dependency the executor omits demotes its transaction to an authorized blind write, never a cross-authority isolation failure. The confinement claimed is database-authority confinement: keeping *authorized* data private additionally requires the egress isolation A12 assumes. The construction's freedom from BFT replication, re-execution, and trusted hardware is inherited from the trusted-broker model rather than contributed by it. We implement Aster in roughly twenty-two thousand lines of Rust; it executes selected unmodified `npx convex deploy` bundles that use the implemented syscall subset. Measured on the real pipeline, the sealing apparatus (IPC round trip, full seal verification, capsule merge, canonical re-MAC) costs ~0.04 ms per authenticated read at small read-sets, scaling linearly at ~0.7 μs per held capsule entry, and a read the capsule already holds costs no trap at all. In a warm-process benchmark, one fenced mutation from inside an untrusted cell (a real JS mutation in a fresh V8 isolate, one authenticated fixture read, commit verb, OCC validation, durable append to Aster's own commit log) completes in 6.5 ms median on stock Postgres (the fence alone: 3.5 ms, 280 serial commits/s); the measured v0.7 pipeline's read adapter and commit log were separate histories, so this prices the full apparatus rather than a live-backend integration; v0.8 has since bound reads, snapshots, retention, and fence to one authoritative history for the standalone Aster plane, a path not yet re-measured. At the measured read-set sizes, the security tax on both paths is a fraction of the storage cost it authenticates.

1 Introduction

Untrusted code next to a real database is now the normal case, not the exception. AI app builders execute model-generated queries and mutations against production data. Agent platforms wire language models to tools that read and write live records. Multi-tenant platforms co-locate customer functions on shared infrastructure. In every one of these settings the security question an auditor asks is the same: *what stops that code from touching data it shouldn't?*

The prevailing answers isolate the compute and guard the credential. Both halves fail in practice, and they fail for the same reason: **the code still holds ambient authority over everything its credential unlocks.** Harden the sandbox all you want; if an environment variable, IAM role, or connection string is reachable from inside it, the isolation boundary is only as strong as the weakest request the code can emit. In September 2025, Sonrai reported driving the AWS AgentCore code interpreter, marketed as "completely isolated," into exfiltrating IAM credentials through the instance metadata service[1]; Unit 42 separately published network-isolation and metadata-service findings in April 2026.[2] A 2025 demonstration showed how a misconfigured Supabase MCP deployment running with the `service_role` credential could be prompt-injected into exposing an `integration_tokens` table: a demonstrated attack scenario rather than a reported customer breach, and the case that popularized the "lethal trifecta" framing (private data, untrusted input, and an exfiltration path in one agent).[3] By 2026 the industry frontier already brokers the *outbound* credential (Anthropic Managed Agents, Cloudflare Code Mode, and Vercel Sandbox all keep long-lived secrets out of the sandboxed code), so credential-free execution is table stakes, not a contribution. The *inbound* half is the gap, and we are not aware of prior work that fills it: authenticate the data the code acted on, and prove at commit time that the code's transaction depended only on real, current rows.

This paper takes that cut. We move the trust boundary off the executor entirely:

- The **executor** (a V8 isolate in its own OS process, one per invocation) is Byzantine from instruction zero. It holds no database credential, no network, no filesystem. Its only channel is a Unix-domain socket to the broker.
- The **broker** is trusted. It owns the Postgres handle, the 32-byte MAC key κ, and the deployment's single-writer lease. It serves reads as **sealed capsules**: keyed-BLAKE3-MACed, canonically encoded bundles binding cell identity, lease epoch, tenant, deployment, snapshot timestamp, the document map, and any range-scan certificates.
- At commit, the executor submits its sealed capsule, a declared dependency set, and a write set. The broker performs classical **OCC backward validation**, but over a read-set whose every entry carries broker-minted provenance. Validation, lease-epoch check, retention check, write policy, and append execute inside one storage-visible **commit fence**.

The key idea in one line: **for isolation you do not need to verify the computation; you need to authenticate the data flow.** Optimistic concurrency control does not need to trust the executor if the read-set is authenticated. FoundationDB's resolver already performs exactly this backward validation over client-declared conflict ranges, while trusting the client entirely; a client that under-declares its reads commits what OCC should have aborted [16]. Aster makes the resolver's input trustworthy: every read-set entry is a serve-time broker MAC, so the executor leaves the trusted computing base.

The result is stated as the **Capsule Transaction Theorem** (Section 4, full proof in the companion technical

[1] https://sonraisecurity.com/blog/sandboxed-to-compromised-new-research-exposes-credential-exfiltration-paths-in-aws-code-interpreters/

[2] https://unit42.paloaltonetworks.com/bypass-of-aws-sandbox-network-isolation-mode/

[3] https://generalanalysis.com/blog/supabase-mcp-blog; https://simonwillison.net/2025/Jul/6/supabase-mcp-lethal-trifecta/

report): under a fully Byzantine executor, read-sets are unforgeable (T1a), the executor's view is confined to its authorized grant transcript (T1b), committed mutations are strictly serializable over the declared, authenticated read/write sets (T2), and, as the honesty boundary, every Byzantine commit is reproducible by an authorized, protocol-following application client with the same grants (T3). A compromised executor is exactly as powerful as a malicious-but-authorized application. It can write garbage *inside its policy envelope*. It cannot fabricate a snapshot observation, read outside policy, evade a conflict on a declared observation, or append through a stale lease epoch.

One sentence survives the proof and organizes the paper:

> Aster makes every submitted OCC observation provenance-authentic without trusting or re-executing the application executor; strict serializability then follows from classical backward validation, while any omitted dependency is demoted to an authorized blind write rather than a cross-authority isolation failure.

1.1 Contributions

- **C1 (mechanism).** Serve-time keyed-MAC capsules that make each read self-certifying, turning classical OCC backward validation into an *online admission gate* against a Byzantine executor. The commit fence couples seal verification, lease epoch, retention coverage, conflict validation, write policy, and append at one horizon (Section 3.4).
- **C2 (theory).** The Capsule Transaction Theorem: read-set unforgeability (T1a), confinement with named leakage (T1b), Byzantine strict serializability (T2), Byzantine equivalence (T3), retention safety (Lemma R), and read-plane scale-out (Corollary C1). The proof is conditional on an explicit assumptions ledger (Table 3) and was hardened by an adversarial review round; it is a protocol specification, not a substitute for code verification (Section 4).
- **C3 (principle).** "Authenticate data flow, don't verify computation," with its precise limit stated as a theorem rather than hidden: under-declaration and capsule rollback demote a transaction to an *authorized blind write* (T3). The guarantee is about **authority, not arithmetic integrity**.
- **C4 (system).** Aster: roughly twenty-two thousand lines of Rust across eight crates, executing selected unmodified `npx convex deploy` bundles (the implemented syscall subset), with commits landing on Aster's own fenced log; since v0.8, the standalone plane serves reads from that same history. Measured on the real pipeline: ~0.04 ms of authentication apparatus per read at small capsules, linear in capsule size (EQ2), and 6.5 ms / 153 tx/s for a warm-process fenced mutation from an untrusted cell: a plumbing benchmark across the v0.7 campaign's two planes (Section 6; the v0.8 single-history path is not yet re-measured).

1.2 What we do not claim

Stating the non-claims up front is part of the contribution's hygiene.

- *"Transactions on untrusted infrastructure without BFT" is not ours.* Fides established auditable transaction management, including atomic commitment over untrusted servers without expensive Byzantine replication, in 2020 [4]. Aster's freedom from BFT replication and re-execution is a *consequence* of placing one trusted broker in the online path, not a novelty. Our delta against Fides is the guarantee class: online **prevention** at a trusted commit gate versus post-hoc **detection** by an offline auditor (Section 7).
- *"The code never holds a credential" is not ours either.* Industry credential brokers made that table stakes by 2026. Ours is the data-plane half: sealed inbound reads and commit-time validation of their authenticity.

- *T2 is not execution correctness.* It is strict serializability over the **declared, authenticated** read/write sets. A Byzantine cell may compute nonsense and lie in its application-level return value; only the broker-served grant transcript and the committed history are covered.
- *Reads may be stale.* The headline consistency guarantee is **strict serializability for mutations; snapshot reads are serializable but possibly stale**, mirroring FoundationDB's own snapshot-read caveat [16].

1.3 Results preview

On the real pipeline (V8 cell, UDS to a long-lived broker, Postgres 16 behind it), the capability apparatus on a read trap (full seal verification of the accumulated capsule, capsule merge, complete canonical re-encode and re-MAC, framed IPC round trip) costs **0.035 ms** at a 1-entry capsule and grows linearly at **~0.7 μs per held entry** (0.70 ms at 1,000 entries): the store's snapshot queries, not the cryptography, dominate a cold authenticated read, and a read the capsule already holds no longer traps at all. (The v0.6 pipeline measured **0.34 ms** per trap on its then-always-trap path; Section 6.1 keeps that baseline and states exactly what changed.) The write path is measured against Aster's own fenced log: a blind commit through the fence costs **3.5 ms** (280 commits/s sustained serial on stock `synchronous_commit=on` Postgres), validation adds one round trip with cost flat in the point-set size up to 200 keys over the tested empty suffix, a conflict answer returns in **1.75 ms** (half a commit, no WAL flush; decision latency, measured before the fence gained its awaited rollback), and the full plumbing loop (capsule issue, real JS mutation in a fresh V8 isolate, commit verb over UDS, Postgres fence) completes one transaction in **6.5 ms** median, **153 tx/s** serial (Section 6.3); the read adapter and the commit log were separate histories in the measured v0.7 campaign (Section 5), so these are apparatus costs, not a live-backend integration. The v0.8 tree has since bound the standalone plane to one authoritative history, not yet re-measured. The cold one-shot floor is spawn plus V8 boot, not capsule machinery: ~390 ms dockerized in the v0.6 campaign, ~6 ms as a bare host process (Section 6.4).

2 Background and threat model

2.1 Target data model

Aster targets the storage model of Convex, an open-source reactive backend: an MVCC document store over a totally ordered commit log, with a **single-writer lease**. Reads execute against an immutable snapshot σ_s at timestamp s; all commits flow through one writer holding the lease; lease **epochs** rise on failover and stale-epoch writers are fenced out at the storage layer (the upstream backend implements this as a lease check inside the Postgres commit path; `crates/postgres/src/lib.rs:1738-1799` in the `convex-backend` repository[4]). This lease is a design constraint we inherit and exploit: active-passive failover per deployment is possible; active-active is not, and Aster never needs it.

2.2 OCC backward validation in one paragraph

A transaction reads at snapshot s, accumulating a read-set R, and submits R with a write set W. The validator picks a horizon h (the log tip), checks that no committed write in the interval (s, h] intersects R, and if so appends W at a fresh timestamp c > h. The classical soundness argument is an induction over the commit order: if every observation in R still evaluates to its snapshot value at the append point, the serial history that orders transactions by commit timestamp is legal. Everything in this paper preserves that classical core; what changes is that **R's entries are no longer taken on faith**.

[4] https://github.com/get-convex/convex-backend

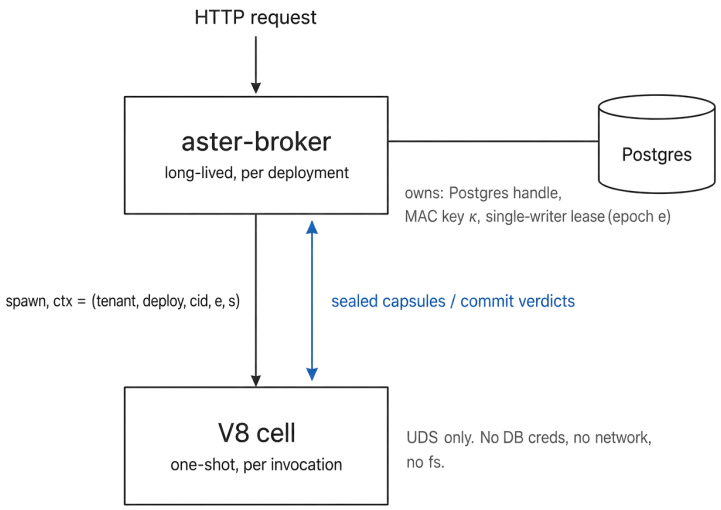

Figure 1: Architecture. The broker owns κ, the storage handle, and the single-writer lease; the cell gets a UDS and nothing else.

2.3 Threat model

Trusted: the storage system (assumed to implement the MVCC and lease contracts; a Byzantine database server is out of model); the broker/committer role; every process holding the capsule key κ. The TCB is broker + storage.

Untrusted: the executor cells: arbitrary PPT Byzantine code from instruction zero. A cell's only channel is the Unix-domain socket to the broker (assumption A12: no database credentials, no network, no filesystem, no alternate storage channel). In deployment this means cells run with egress blocked, and the shipped v0.8 production profile enforces exactly this (cells with no network namespace, read-only root filesystems, all capabilities dropped, kernel peer-credential admission on the socket; Section 5); without it, exfiltration of *authorized* data is trivial and the confinement pitch collapses even though the theorem survives.

Collusion: cells may share out of band anything each was authorized to read; no protocol prevents authorized principals from pooling their authority. What the protocol does prevent is *relabeling*: channel binding stops one cell from replaying another live context's capsule on its own channel (Section 3.2).

Out of scope, stated up front: liveness against Byzantine cells (a cell can always self-abort or spam retries), denial of service and resource exhaustion, timing/cache/speculation side channels, exactly-once execution (a replayed request that revalidates is two serial transactions), and compromise of a key-holding broker.

Goal. Strict serializability of committed effects plus confinement of the executor to its authorized policy envelope, achieved *online*, at the commit gate, with no trusted hardware and no replication.

3 Design

3.1 Architecture

A long-lived broker runs per deployment; it owns κ, the storage handle, and the lease. Each invocation gets a fresh one-shot cell: a V8 isolate in its own OS process, connected to the broker by a Unix-domain socket (`crates/ipc/src/bin/aster_brokerd.rs`, `crates/ipc/src/bin/aster_v8cell.rs`). The broker spawns the cell with a context

ctx = (tenant, deployment, cid, e, s)

where *cid* is the cell identity, *e* the lease epoch, and *s* the per-invocation snapshot timestamp. After the initial grant, the context is **not** cell-asserted: every capsule verb resolves its expected context exclusively from the broker's own session table (repair C-CHANNEL, Section 3.2), so a request-supplied context is either omitted or must equal the registered one. Since v0.8, mint-time identity is no longer a bare request claim either: a Postgres-mode broker refuses `InitialCapsule` without a one-time launch token, broker-keyed, TTL-bounded, spent atomically on first use, and binding (cell, tenant, deployment, authority epoch) (`crates/ipc/src/launch.rs`); what A11 still assumes is the integrity of the token issuer and the supervisor chain that launches cells (Sections 3.2, 5). Every *document* the cell ever sees arrives as (part of) a sealed capsule (module bundles are broker-provided launch inputs outside the capsule theorem, their authorization and confidentiality not covered by T1b; Section 8), and the cell never holds a credential of any kind.

3.2 Capsules and the seal

A capsule is

Cap = (tenant, deployment, s, docs, ranges)

where `docs` is a finite map from document keys to versioned results, including **explicit absence** (a versioned "no such document" and a versioned tombstone are first-class observations, because a mutation that depended on absence must conflict when the document later appears), and `ranges` is an ordered sequence of range certificates (Section 3.5).

Canonical encoding. The capsule has exactly one wire representation: length-prefixed strings, fixed-width little-endian integers, maps in strict key order, tagged sum types, and a fixed domain string (`aster-capsule-v3\0`) at offset zero (`crates/capsule/src/canon.rs`). Injectivity of this encoding is proved by exhibiting a deterministic decoder (Lemma 3.1 of the technical report). Just as important, the production decoder is a **canonical decoder, not a permissive deserializer** (repair W-CANON): it rejects duplicate keys, out-of-order keys, invalid tags, non-minimal forms, truncated input, oversized declared lengths, and trailing bytes, accepting exactly the byte strings the encoder can produce. A permissive "last duplicate wins" parser would let attacker-controlled bytes carry two meanings across parsing and MAC recomputation; the codec's test suite drives 14 adversarial-decode rejections through exactly those corners (18 codec tests in all; the other four pin positive round-trips and canonical ordering). On the JSON IPC path, the same structural validation is enforced at the seal-verification chokepoint (`SnapshotCapsule::validate_structure`, called from `SealedCapsule::verify`), where every capsule must pass.

The seal, and an honest version history. The first production seal (algorithm string `aster-blake3-keyed-v1`) MACed a BLAKE3 *digest* of the canonical bytes: a prehash. Counterexample 2.2 of the technical report shows why that construction cannot be reduced to keyed-MAC security alone: literal injectivity of a 256-bit digest over unbounded capsules is impossible by counting, and there is a proof-theoretic countermodel in which the

keyed mode is a perfect PRF while the unkeyed hash maps every capsule to one constant: a tag for one capsule would then verify for every capsule sharing its context. Not a practical attack on BLAKE3; a demonstration that the v1 proof needs an *extra* assumption, collision resistance of unkeyed BLAKE3 (ledger entry A3). The v0.7 cycle therefore moved to direct MACing in two steps: `aster-blake3-keyed-v2` MACed the full framed canonical bytes, retiring A3 (Remark 3.4 of the report); the shipped seal, `aster-blake3-keyed-v3`, keeps the direct-MAC construction and additionally binds the broker session into the MAC input (the channel-binding repair below), superseding v2 within the same cycle. The v3 MAC input is

alg ‖ lp(cid) ‖ le64(e) ‖ SB ‖ lp(E(Cap)), SB = 0x00 (unbound) | 0x01 ‖ session[32] (bound)

This is the **full framed canonical encoding**, with tenant, deployment, and snapshot bound through E(Cap), which frames them immediately after the domain string, and SB a domain-separated session frame whose tag byte alone determines the frame length, so a bound message can never collide with an unbound one (`crates/capsule/src/seal.rs::seal_mac`). BLAKE3 [8] is a streaming hash, so MACing the full encoding costs no second materialization. Forging any accepted capsule reduces to a keyed-MAC forgery alone; assumption A3 stays retired. The canonical digest is still computed and carried in the seal, but only as an audit and tooling convenience, never a MAC input. Verification enforces the exact algorithm identifier (only v3 verifies; v1 and v2 seals are rejected: `seal.rs::sealed_capsule_rejects_legacy_v1_algorithm`, `sealed_capsule_rejects_legacy_v2_algorithm`), exact 32-byte tag length, canonical structure, header/context equality, and constant-time tag comparison (`crates/capsule/src/seal.rs::ct_eq`). Two pinned test vectors guard the wire format, one per session state: any drift in the seal construction is a deliberate, versioned decision, never an accident (`seal.rs::seal_test_vector_is_stable`, `bound_seal_test_vector_is_stable`).

Channel binding (seal v3). Binding the MAC to *cid* means "wrong-cell rejection" only if the verifier knows which cell a request came from by a channel it controls. v0.7 implements C-CHANNEL as **per-attempt bearer-capability binding**: what is verified is possession of an unguessable broker-minted session id, not a trusted physical-cell identity. At `InitialCapsule` time the wire broker draws an unguessable 32-byte session id from OS entropy, registers it in its own session table, and returns it with the grant (`InitialCapsuleGrant`, `crates/ipc/src/bin/aster_brokerd.rs`). Every subsequent capsule verb must present that id, and the broker rebuilds the expected bound context **exclusively from its own table entry**; the request's serialized context is checked for equality against the record and then discarded, never used as authority. The enforcement point is the seal itself: the session enters the v3 MAC input under domain separation, so a capsule issued to one session and presented on another (even with identical *cid* and epoch, the re-spawned-cell case) fails seal verification (`capsule_from_another_session_fails_seal_verification`), and stripping the session field from a bound seal dies on the MAC's tag byte. A capsule plus its public context copied to another cell's channel therefore fails verification, while a *cooperating* holder of the other context can of course exercise its own authority, which is collusion the model explicitly permits. One prototype caveat, stated rather than blurred: the session is per-invocation, not per-socket; the prototype wire protocol opens one connection per request, so the unguessable id *is* the channel. Mint-time identity, an open obligation through v0.7, is closed in v0.8 by both mechanisms the earlier text named: the broker's accept path enforces kernel `SO_PEERCRED` UID admission on the UDS, and a Postgres-mode broker refuses to mint a session without a **one-time launch token**: issued under a separate launch key, TTL-bounded, spent atomically on first presentation, and binding (cell, tenant, deployment, authority epoch), so a stale pre-failover or hostile launch claim dies at mint (`crates/ipc/src/launch.rs`; `launch_token_required`, `stale_lease_epoch`). A hostile cell can no longer mint a session under an arbitrary label. What remains assumed rather than proved is the chain above the token: A11 in Table 3 now names the token issuer and the container supervisor as the trusted launch authority.

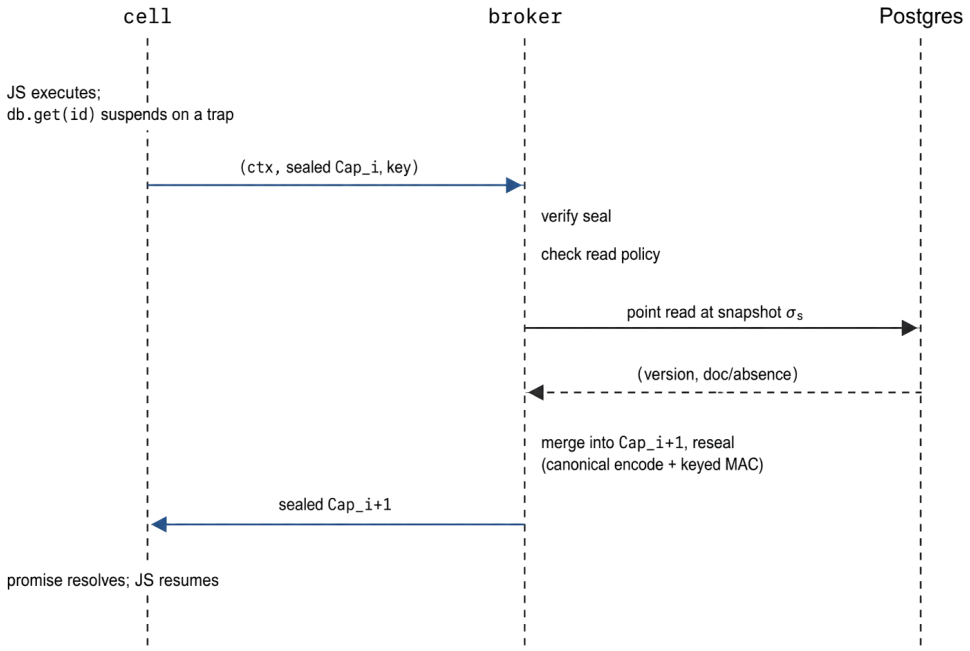

Figure 2: One read trap.

3.3 The read protocol: grow-and-reseal

Reads are pull-based **traps**. The cell starts with a sealed initial capsule (possibly prewarmed: every prewarmed item passes the same read-policy check a trap would, and enters the grant transcript; prewarm is an ideal read grant, not a policy bypass). When the JavaScript touches a key that is not in the capsule, the runtime suspends, sends the sealed capsule and the request over the UDS, and the broker: verifies the seal against the channel-bound context, checks read policy *before* any storage access, evaluates the point read or scan at the immutable snapshot σ_s, merges the exact result, and reseals the grown capsule. Denied requests return a denial code and nothing else.

Two properties matter. First, the broker is **stateless per invocation**: it remembers no "latest capsule" digest, no read list, no sequence number, only immutable launch metadata and global state (policy, lease, log, key). This is what lets read brokers scale out (Corollary C1) and it is also what makes the honesty boundary of Section 4 exactly what it is: because the broker accepts any *issued* capsule, a Byzantine cell can replay an earlier one and fork its own issuance tree. Second, **absence is authenticated**: a read of a missing or tombstoned key enters the capsule as a versioned atom, so "I saw nothing there" is as unforgeable (and as conflict-checked) as "I saw value v."

3.4 The commit protocol and the commit fence

At commit the cell submits (ctx, sealed Cap, S, W): its channel-bound context, a sealed capsule, a **declared dependency set** S (Variant B, Section 3.6), and a canonical write set W. The broker reduces this to the theorem's validated ingredients and executes the **commit fence**:

The load-bearing property is that **validation and append share one stable horizon** (repair A-ATOMIC). The classical write-skew pair shows why a check-then-append implementation is simply wrong, not merely

```
CommitFence(ctx, Cap, R, W):
  enter a storage-visible commit/validation serialization fence
  require the committer's epoch e_now was acquired and fenced at storage
  read log tip h only after that epoch-acquisition fence
  read monotonic wall/log time Now and retained low-watermark g
  require ctx.e == e_now                      // V2  (stale-epoch fencing)
  require ctx.s <= h                          //     (snapshot is a real prefix)
  require ctx.s >= Now - Delta                // V3  (product max-age rule)
  require g <= ctx.s                          //     (actual coverage rule)
  pin the retained interval so g cannot advance past ctx.s
  evaluate P.write(ctx, W) against policy version p      // V5
  require no committed write in (ctx.s, h] intersects
      any point/window in R                   // V4  (backward validation)
  atomically append W at fresh c > h iff:
      the storage lease still accepts ctx.e, and
      the policy decision/version p is still current
  release the fence and retention pin
```

Figure 3: CommitFence pseudocode (technical report §1.8).

racy: T1 reads y=0 and writes x=1; T2 reads x=0 and writes y=1; both validate against the same tip h=s, both see no conflict, both append, and no serial order realizes both reads (Counterexample 3.9 of the report). Serializing each validation with its append forces whichever transaction reaches the fence second to see the first writer in (s, h] and abort.

Our implementation maps the fence onto **one Postgres transaction** (`crates/store-postgres/src/write_plane.rs::com` the first statement locks the deployment's lease row `FOR UPDATE` (every commit takes that lock, so validation and append serialize); V2 requires *both* the committer's acquired epoch and the capsule context's epoch to equal the live lease epoch (a current committer still rejects a stale-epoch capsule); the horizon h is read only after the epoch fence; the retention row is locked `FOR UPDATE` from the coverage check $g \le s$ through append, so the GC sweeper, which takes the same lock, cannot remove an event in (s, h] mid-validation (Lemma R's pin); the conflict scan checks declared point keys and range windows against committed writes in (s, h]; and the append at c = h+1 carries the epoch inside the same transaction. A concurrent lease acquisition blocks on the same row lock until an in-flight fence commits, which is precisely the failover ordering Lemma 3.11 requires: an old-epoch append linearizes before the new epoch's first fence, and every later old-epoch fence fails the epoch equality check. Lease epochs are **strictly increasing and never reused**: acquisition always bumps the epoch under the row lock, even across an A→B→A failback; epoch reuse would break the exhaustive case analysis of the epoch-fencing lemma. This is the same lease-in-the-commit-path pattern the upstream Convex backend uses for its single writer; the fence adds validation, retention, and policy to the decision that storage already serializes. Coverage ($g \le s$) is the exact admission condition implemented in the fence; the wall-clock max-age rule ($s \ge \text{Now} - \Delta$) is a product admission policy layered above it.

3.5 Phantom-safe limited scans: range certificates

Point reads are not enough: `db.query(...).take(5)` must be phantom-safe. A limited scan's result is protected by a **sealed range certificate**:

ρ = (I, direction, ℓ, ⟨k1 … km⟩, stop), stop ∈ {Exhausted (m < ℓ), Boundary (m = ℓ)}

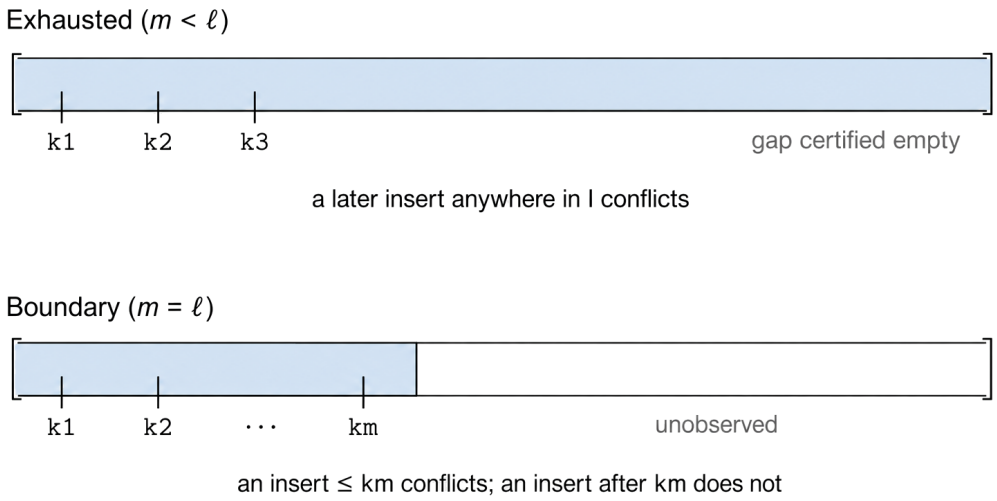

Figure 4: The two conflict windows of a limited ascending scan.

binding the normalized interval with endpoint inclusivity, the scan direction, the positive limit, the ordered returned keys, and, critically, whether the scan **stopped at the limit** or **exhausted the interval** (repair R-RANGE). Every returned key must cross-reference an exact entry in the capsule's document map. The certificate determines the **observed window** that commit-time validation defends:

A full result protects the prefix through the last returned key; a short result certifies an **exhaustion gap** (no additional live key existed at s) and protects the whole interval. This is next-key locking's insight transplanted into OCC validation windows. Stability is proved as Lemma 3.7 of the report: if no committed write in (s, h] touches Win(ρ), the first-ℓ answer at h equals the answer at s, including versions and documents.

One API consequence deserves the ink (finding F2 of the adversarial review): when an application asks for `limit` ℓ and receives exactly ℓ results, it is protected as a *first-ℓ* observation, **not** as a completeness assertion. Correctly so: an insert after k_ℓ does not change the first-ℓ answer. An application that wants "this is the complete set" must obtain an Exhausted certificate: ask for ℓ+1, or check the stop bit. We document this in the API and repeat it here because it is exactly the kind of semantic footgun that survives type checking.

The certificate types, their window computation, and the window-containment predicate used by the fence live in `crates/capsule/src/lib.rs` (`RangeCertificate`, `ScanStop`, `ObservedWindow`); the canonical encoding includes the ordered certificate sequence (capsule domain `aster-capsule-v3`); v0.7 serves certified prefix scans over the wire protocol, so a scan's certificate is sealed into the capsule the cell holds, not reconstructed at commit.

3.6 Variant B: declared dependency sets

At commit, which observations does validation defend? Variant A validates the *entire* submitted capsule: R = Obs(Cap). Variant B validates an exact **declared subset**: R = S ⊆ Obs(Cap), where each declaration

references a sealed atom precisely: a point declaration must match the sealed key and exact versioned result; a range declaration must identify an exact sealed certificate entry, never a narrowed or rewritten interval; duplicates and atoms not structurally present in the capsule are rejected (repair B-SUBSET).

v0.7 adopts **Variant B**, and the reason is worth being precise about, because A's apparent advantage is illusory. Variant A looks stronger ("the committer does not accept omission"), but it proves completeness only relative to *whichever issued capsule the adversary chooses to submit*. Under a stateless broker, a Byzantine cell can compute on a value from a later capsule and submit the still-valid seal on an earlier one (Counterexample 2.1); no stateless verifier can distinguish that from an application that stopped early and issued a blind write. So A buys no Byzantine completeness, while costing honest users real aborts: every unused prewarmed atom in the selected capsule participates in conflict checking, so prewarming converts prediction error into abort amplification. Variant B has the same T2/T3 theorem class, avoids false conflicts on unused prewarm, and names the object honestly: S is the *declared dependency set*, not "the executor's complete read set."

For honest code, the runtime does the declaring. The v0.7 V8 runtime tracks **consumption**, not just traps: every observation the JavaScript actually touches (including warm hits served from the capsule without a trap) lands in a deduped consumption ledger surfaced as `V8ExecutionResult::consumed_reads`, and prewarmed entries stay out of it until consumed. That ledger is S; the cell-facing commit verb submits it over the wire, and the broker enforces the declared set against the sealed capsule (every declared key must be an observation the capsule carries: B-SUBSET) before anything reaches the fence (Section 5). Honest applications therefore get ordinary OCC behavior with no bookkeeping, and the declared set equals actual dependencies whenever the runtime is intact. When it is not (a compromised runtime can omit), T3 bounds the damage: the omission demotes the transaction to an authorized blind write. One shipped narrowing (re-referee F6): v0.7 implements Variant B **for point observations** and conservative whole-capsule validation for range certificates: the committer converts *every* sealed range in the capsule into a conflict window, declared or not. That is safe (conservative) but hybrid: exact declared range subsets, and with them the no-false-conflict prewarm claim *for ranges*, are not yet implemented, so an unused speculative scan still amplifies aborts.

4 Formal guarantees

This section condenses the companion technical report ("The Capsule Transaction Theorem", 27 pp., whose proof survived an independent adversarial review round with no fatal findings) into the four theorems, the retention lemma, and the scale-out corollary, with an emphasis on what each statement does **not** claim (Table 2). The proof is **conditional** on the assumptions ledger reproduced as Table 3; it is a specification for v0.7, not a claim that any particular code revision is verified.

T1a (read-set unforgeability). Assume the keyed MAC is EUF-CMA-secure, the canonical encoding and outer framing are injective (proved as Lemmas 3.1/3.2, not assumed), verification performs the exact canonical and context checks, and all key holders are trusted. Then for every PPT coalition of cells, the probability that the verifier accepts a capsule **not issued for that exact channel-bound context** is bounded by the MAC-forgery advantage plus a negligible term. (For the retired v1 prehash seal, the bound carried an additional unkeyed-BLAKE3 collision-resistance term; the direct-MAC construction, v2 and the shipped session-bound v3, removes it.) Consequences: no cross-context transplant (different cid, epoch, tenant, deployment, or snapshot changes the MAC input), no splicing entries from two capsules into an unissued union, no payload substitution. **Honest scope:** T1a authenticates *an issued* capsule, not "the latest." Replay of an earlier issued capsule under the same context is deliberately outside the word "forgery"; preventing it would require stateful anti-rollback or trusted use-observation, both of which the stateless design refuses. This scope is not a proof convenience; it is load-bearing for the honesty boundary in T3.

T1b (confinement, with named leakage). The coalition's read-plane view is computationally indistinguishable from a simulation given only the public contexts and the ideal grant transcript: it learns no document payload beyond the adaptive transitive closure of its policy-authorized reads. The *whole-protocol* view (once commits are visible) additionally leaks explicit control predicates (epoch/retention/policy outcomes and, in particular, the **conflict bit**: whether a declared authorized observation window changed after the snapshot). Counterexample 2.3 shows the bit is real (two executions identical up to an unrelated post-snapshot write differ in commit outcome), so the ideal functionality names it rather than pretending it away. No unauthorized payload is revealed either way. Timing, cache, and scheduler channels are excluded by assumption, not hidden by the theorem.

T2 (Byzantine strict serializability). Except with the negligible T1a failure probability, every execution (arbitrarily many Byzantine cells, arbitrary interleavings, retries, and lease failovers) yields a committed-mutation history that is strictly serializable over the **declared, authenticated** read/write summaries (R_T, W_T, s_T, c_T), in commit-timestamp order, with the atomic append as the linearization point; real time is respected between non-overlapping successful operations. The proof is the classical backward-validation induction, with the point/range stability lemmas as the semantic bridge and the fence lemmas (single horizon; epoch block order) closing concurrency and failover. Read-only invocations get a serializable snapshot that may be **stale**; their application-level return value is not authenticated at all; only the broker-served grant transcript is. *Versus FoundationDB:* the resolver performs the same backward validation over client-declared ranges but trusts the client; Aster's proven delta is **provenance authenticity of each submitted observation**: the executor cannot fabricate a value, version, or sealed window. The stronger sentence "the server now knows the complete actual read set" is false for the stateless protocol, and the report says so in exactly those words.

T3 (Byzantine equivalence, the honesty boundary). For every successful Byzantine-produced commit there exists an *authorized, protocol-following* client (honest about wire syntax, broker issuance, and policy, not about application semantics) with the same context, the same grant transcript, and the same commit-time write authorization, that submits the identical accepted summary. The witness may hold W as a literal constant and treat any grant as unused; rollback and Variant B omission are reproduced by the witness *by design*. If write policy confines each tenant to disjoint key authority, then compromising the executor yields **no database effect beyond a malicious-but-authorized application**: garbage writes, blind writes, retries, self-conflicts, all inside the policy envelope; no unauthorized reads, no out-of-policy writes, no forged observations, no serialization break. This is the security pitch, stated as a theorem. Its honest limit: omission can destroy the malicious tenant's *own* semantic invariants. It creates no cross-tenant or cross-policy power.

Lemma R (retention safety). Backward validation is sound iff the consulted log covers (s, h]: sufficiency is the stability lemmas; necessity is a concrete truncation counterexample (read k at s; another commit changes k; GC removes the event; the stale read validates). The exact admission condition is coverage (retained low-watermark $g \leq s$, held under a retention pin so GC cannot advance past s mid-validation), with the age rule $s \geq \text{Now} - \Delta$ as the product's uniform admission bound. Liveness corollary: an invocation whose snapshot ages past Δ has no commit guarantee and must retry from a fresh snapshot.

Corollary C1 (read-plane scale-out). Any trusted κ-holder that can read the fixed snapshot and enforce read policy can verify, grow, and reseal capsules; only the single committer runs the fence and appends. Read brokers therefore scale horizontally without changing T2: the committer re-verifies every capsule against the authoritative history; a read broker with stale epoch information at worst produces a capsule whose commit later aborts. One deployment caveat (review finding F3): read policy is evaluated per-broker at read time, so **revocation propagates with bounded skew** across scaled read brokers; operators must bound and monitor that latency. A second caveat is v0.7-specific (re-referee F4): **C1 is an architectural result, not an implementation claim.** The shipped broker binds sessions in an in-process table (a capsule minted by broker A cannot be presented to broker B, sticky routing at best) and always acquires the writer lease at boot

in Postgres mode, so launching a second broker advances the epoch and invalidates the first broker's sessions. A read-only broker mode and a shared session authority are named future work; session consumption at Commit is atomic (the session is spent before the fence runs).

Table 2: theorem map (what each result does and does not claim).

Result	Guarantees	Explicitly does NOT claim
T1a	An accepted capsule was issued by a trusted broker for exactly this channel-bound context; no transplant, splice, or substitution	That the capsule is the *latest* issued; that it contains everything the cell actually used (rollback/fork permitted)
T1b	Read-plane view simulatable from the authorized grant transcript; whole protocol adds only named control bits (conflict bit, policy/epoch/retention outcomes)	Hiding of policy allow/deny outcomes; anything about timing/cache side channels
T2	Committed mutations strictly serializable over declared, authenticated read/write sets, in commit order; append is the linearization point	Execution correctness; completeness of R w.r.t. actual reads; freshness of read-only snapshots
T3	Every Byzantine commit reproducible by an authorized protocol-following client with the same grants: compromise ≡ malicious authorized app	Application-semantic honesty; protection of a tenant from its own authorized code
Lemma R	Validation sound iff retained log covers (s, h]; $g \leq s + \text{pin}$ is exact; Δ-age rule is the uniform product bound	Commit liveness for snapshots older than Δ
C1	Read brokers scale without a new serialization point; committer re-verifies everything	Instantaneous policy-revocation propagation across read brokers (bounded skew)

The report additionally discharges **thirteen enumerated attack obligations** (capsule replay across snapshots, cross-cell transplant, cross-epoch replay racing failover, splicing/rollback/fork, phantoms including the exhaustion gap, absence flips, encoding ambiguity, Variant B under-declaration, over-declaration/prewarm poisoning, GC races, duplicate/contradictory entries, policy TOCTOU, and whole-capsule replay), absorbing rollback and under-declaration into the theorem boundary rather than pretending to exclude them.

Table 3: assumptions ledger (condensed from the report; P-entries are proved, not assumed).

ID	Assumption / proved fact	Role
A1	Trusted broker & key holders	Every κ-holder enforces policy, reads the requested snapshot, seals only truthful transitions; compromise defeats T1a/T1b
A2	Keyed BLAKE3 is EUF-CMA / PRF	The MAC in T1a/T1b
A3	Unkeyed BLAKE3 collision resistance	Required only by the retired v1 prehash seal; **retired in v0.7** by the direct-MAC seals (v2, superseded in the same cycle by the shipped session-bound v3)
A4	Key secrecy & domain discipline	κ from a secret store, not public environment material; cross-protocol uses get disjoint tags or keys
P1	Injective canonical encoding (**proved**, Lemma 3.1)	Requires the production decoder to reject noncanonical/duplicate forms
P2	Injective outer framing (**proved**, Lemma 3.2)	Exact algorithm prefix; framed/fixed-width fields
A5	Correct MVCC storage	Snapshot reads immutable; tombstones/absence as modeled; appends atomic & durable
A6	Complete conflict projection	Every mutation that can change a point or key-interval scan surfaces as a write event on the corresponding key: **the weakest joint** (Section 8)
A7	Single-writer lease; strictly increasing, never-reused epochs	Storage rejects stale epochs atomically; assigns globally increasing timestamps
A8	Atomic commit fence	V2–V5 + append at one stable horizon; no interposing commit
A9	Retention floor & pin	Committer knows g; GC cannot remove (s, h] during validation
A10	Policy correctness & versioning	P.read/P.write express intended authority; write decisions linearized with append; tenant confinement needs disjoint key authority

ID	Assumption / proved fact	Role
A11	Context/channel binding	cid, tenant, deployment, epoch, snapshot from trusted launch/channel metadata, never cell assertions. **Narrowed in v0.8**: post-mint verbs resolve context from the broker's session table; mint requires a one-use launch token binding (cell, tenant, deployment, epoch) plus `SO_PEERCRED` UID admission on the socket; the residual assumption is the integrity of the token issuer and supervisor chain (Sections 3.2, 5)
A12	Process isolation	Cells: no DB credentials, no network, no filesystem, no alternate channel; side/covert channels excluded; enforced by the shipped v0.8 production profile (rootless, read-only rootfs, all capabilities dropped, `network_mode: none` cells, resource limits; Section 5), not independently audited
A13	PPT adversary	All cryptographic conclusions computational

5 Implementation

Aster is roughly twenty-two thousand lines of Rust (22.1k measured by `wc -l` over `crates/`, tests and the S10 bench harness included) across eight crates: `capsule` (canonical codec + seal), `broker` (cell-facing capability trait + store abstraction + the `CommitFence` seam), `store-postgres` (Convex-schema read adapter, the Aster write plane, and, since v0.8, the authoritative single-history store), `v8cell` (V8 isolate, ESM module loader, Convex read and mutation shims), `ipc` (UDS framing, the `aster-brokerd` daemon and `aster-v8cell` one-shot binaries, the launch-token and deployment-policy modules with the `aster_launch_token` issuer, the S10 bench bin), `convex-codec` (IDv6 + ConvexValue ports), `runner`, and `host` (test harnesses and the legacy toy-runner bench). The tree carries 287 tests (grep-counted `#[test]`/`#[tokio::test]`, the Postgres-gated integration lanes included); the seal, codec, and range-window properties of Section 3 are each pinned by unit tests, and the fence's concurrency claims by the integration suite below.

Runs unmodified Convex bundles. The cell compiles a real `npx convex deploy` bundle as an ES module and drives the upstream wire shape, `Convex.asyncSyscall("1.0/get", argsJson)`, through the trap loop (`crates/v8cell/src/lib.rs`): one trap per read the user's query actually makes. The Postgres adapter reads the *same* schema the upstream Convex backend writes (`documents`, `_tables`, `_modules`, `_source_packages`), including the `_modules` × `_source_packages` join and ZIP unpack that resolves module source (`crates/store-postgres/src/{lib,module_index,modules_storage,table_mapping}.rs`).

No application rewrite: the same bundle that deploys to a Convex backend runs in an Aster cell.

The v0.7 write path. What this paper's protocol required over the read-only prototype, and what shipped:

- the direct-MAC, session-bound seal (`aster-blake3-keyed-v3`; the session-less direct-MAC v2 was superseded within the cycle and is rejected alongside v1) with constant-time comparison and two pinned wire vectors (`crates/capsule/src/seal.rs`);
- the canonical wire codec with adversarial decode (`crates/capsule/src/canon.rs`, 14 rejection tests among its 18);
- sealed range certificates with exhausted/boundary windows, served over the wire (`crates/capsule/src/lib.rs`);
- session-bound capsule verbs (C-CHANNEL): a broker-minted unguessable session id per invocation, resolved against the broker's own session table and enforced in the seal MAC (Section 3.2); mint-time identity from trusted launch metadata remains a named obligation below; lease epochs are authority-derived at the broker since S9 (a mint-time context claiming any other epoch is refused), with delivery to the cell still riding the launch environment;
- Variant B consumption tracking in the V8 runtime (Section 3.6), which also closed a real bug the first benchmark exposed: the production syscall path never short-circuited a warm capsule, so every read trapped even when the document was already sealed into the capsule;
- the **write plane**: lease authority and commit fence as one Postgres transaction over Aster-owned tables (`crates/store-postgres/src/write_plane.rs`), plus GC (`advance_retention`) serialized against in-flight fences by the retention row lock;
- the **cell-facing write path** (S9): `Commit`/`Abort` verbs on the UDS protocol, where the broker resolves the session, verifies the capsule seal, enforces the declared set against the sealed observations (B-SUBSET), derives conflict windows from every sealed certificate (never from a cell claim), and drives the `CommitFence` seam; any structured commit/abort answer closes the session (one session = one transaction attempt); and the V8 mutation syscalls (`1.0/insert`, `1.0/shallowMerge`, `1.0/replace`, `1.0/remove`) that grow the write set inside the cell with read-your-own-writes semantics and no store authority (`crates/ipc/src/{lib.rs,bin/aster_brokerd.rs}`, `crates/v8cell/src/lib.rs`). In postgres mode the broker's epoch comes from `acquire_lease` at boot and is stamped into every minted session; a mint-time context claiming any other epoch is refused.

The fence and its concurrency claims are exercised by eighteen integration proofs against real Postgres (`crates/store-postgres/tests/write_plane_it.rs`): lease epochs strictly increase and never reuse across failback; the CE 3.9 write-skew pair is impossible sequentially **and under real concurrency** (two racing fences, exactly one commits); a stale epoch cannot append after failover; the Lemma R retention pin holds in **both directions** (the GC sweeper blocks on an in-flight fence and, conversely, a fence blocks while the sweeper side holds the retention lock, releasing into a successful commit); a wedged idle lock-holder is killed by the configured idle-in-transaction timeout so failover resumes instead of hanging forever; a replayed request commits as a second serial transaction; a phantom insert conflicts with an exhausted window but not past a boundary window (the F2 negative case); absence and tombstone reads conflict on later writes, and a tombstone **write** landing inside the window conflicts with a point read of that key; MVCC point/prefix reads follow snapshot semantics; and the retention watermark clamps to the log tip (`retention_watermark_clamps_to_log_tip`, the regression for the liveness find below); the plane's own MVCC read API refuses snapshots below the retention floor (`reads_below_the_retention_floor_are_stale`, closing on the write plane's reads the same false-evidence class the capsule store's floor guard closed); the scan order is pinned bytewise against Rust's (`prefix_scan_order_is_bytewise`, the F7 collation contract); a stranded above-tip floor is repaired at lease acquisition (`retention_floor_above_tip_is_repaired_at_lease_acquisition`, the R0 discharge); and a parity proof drives one identical multi-step scenario through both `CommitFence` implementations (`memory_fence_matches_write_plane_outcomes`), so the database-free suite's fence

semantics cannot drift off the real plane. Beyond testing, the fence's abstract design is model-checked in TLA+ (`tla/AsterFence.tla`, runs recorded in `tla/RESULTS.md`), with the scope stated exactly (re-referee F10): TLC checks the safety consequences of a fence whose lease-row serialization is *assumed*; it does not verify that Rust and Postgres establish those guards; the SQL integration tests carry that weight. The model's teeth are its mutants, each required to fail: epoch reuse violates I3, dropping the retention pin violates I2, and removing fence atomicity (`AsterFenceNoAtomic`, two fences validating against the same captured horizon over the faithful `(pos, key)` primary key) reproduces exactly the Counterexample 3.9 write skew, the class of bug that survives example-based tests. That investment has already paid for itself: building the model exposed a real commit-admission **liveness** bug before any deployment: `advance_retention` accepted a watermark above the log tip, and since the retention floor never lowers, every subsequent snapshot would fail coverage forever, permanently wedging admission. The fix clamps the applied watermark to the tip under the same retention row lock, with the regression test above, a matching model guard (`w <= Tip`), and all four TLC configs re-run: the positive model clean, both negative models still producing their designed violations.

The committer-integration decision (F9). A write plane cannot be a sidecar: the single-writer lease (A7) forbids two writers, so for a live Convex deployment the Aster broker must either (a) *become* that deployment's committer, taking over the lease and the write path, or (b) validate and *forward* authenticated writes through the backend's own committer, acting as a gate in front of it. **Neither takeover of a live Convex deployment is built.** The shipped write plane owns its own commit log, lease, and retention tables (schema `aster`) and proves the fence against them; it never writes Convex's tables. What v0.8 *did* build is the single history within Aster's own plane: the Postgres-mode broker now constructs its capsule store over the same write plane the fence appends to (`AuthoritativeCapsuleStore`, `crates/store-postgres/src/authoritative.rs`). Snapshot selection, document reads, retention, conflict validation, and append all run against one `aster.log`, with the lease epoch acquired at boot and `ASTER_LEASE_EPOCH` ignored in that mode. The end-to-end proof commits a mutation and reads it back from a *subsequently launched* cell on the same history (`crates/ipc/tests/authoritative_postgres.rs`); the production Compose smoke drives the same loop through a real Convex bundle. The Convex-schema read adapter remains for module resolution (a deployment input) and the historical benchmarks. Integration with a live Convex deployment's committer (option (a) or (b) above) remains a scoped-out product decision, stated rather than blurred.

The plumbing loop is closed; the measured campaign's two planes were separate histories. Since S9, mutations travel the path the theorem describes: JavaScript in the untrusted cell issues mutation syscalls that build the write set in-cell, the consumption ledger becomes the declared set, and the `Commit` verb carries (sealed capsule, declared reads, write set) over the UDS socket to the broker, which reduces it to a `FenceInput` and lets the Postgres fence decide. The gated end-to-end proof commits a JS-born write set through the real fence and aborts an interleaved conflicting cell (`pg_v8_mutation_write_set_commits_and_interleaved_conflict_aborts`, `crates/ipc/src/bin/aster_brokerd.rs`); the write-path benchmark this unlocked is Section 6.3, and its harness is versioned in-repo (`bench/run-v07.sh`, `crates/ipc/examples/bench_v07.rs`, results committed under `bench/results/v07/`). How to read this, precisely (re-referee F1): in the *measured v0.7 campaign* the read plane served the Convex schema and the fence appended to the independent `aster` schema (two histories, a write on one plane invisible to the other), so that proof and the Section 6.3 benchmark exercise the cell→broker→fence *apparatus*; they do not instantiate T2 end-to-end. Since v0.8, the standalone plane closes exactly that seam (previous paragraph): reads and fence share one authoritative history, which is the configuration T2's single-history premise names; not yet re-measured. T2 applies to the standalone Aster plane, and conditionally to a future live-committer integration (the F9 decision above).

The v0.8 closure. After the v0.7 campaign, four product blocks landed, each with tests; the evaluation numbers of Section 6 predate all of them. (i) The **authoritative single history** (the F9 para-

graph above) and the **launch-token mint identity** (Section 3.2). (ii) **Deployment policy as an artifact**: an explicit JSON policy with independent read/write/scan/module prefix allowlists, per-table insert gates, and transaction/session limits, wildcard only when explicit and deny-all in the generated skeleton (`crates/ipc/src/policy.rs`, `docker/policy.production.example.json`). (iii) **Real-bundle mutations end to end**: `db.insert`/`db.patch`/`db.replace`/`db.delete` from unmodified deploy bundles with read-your-own-writes, and server-side canonical IDv6 allocation (the broker consults the deployment's `_tables` mapping and OS entropy and returns the canonical id; a cell cannot self-mint one), proven by the production smoke, which executes a real bundle's query and mutation, commits through the fence, and reads the document back from a fresh cell (`docker/smoke-bundle.sh`). (iv) The **hardened operational profile** of the obligations paragraph below. None of these change the protocol of Sections 3–4; they discharge implementation obligations the earlier draft could only record.

Engineering findings from the first real benchmark. (1) The broker's connection budget (`ASTER_MAX_CONNECTIONS`) was enforced as a *lifetime* counter: the daemon counted total connections since boot and exited when crossed; with one connection per trap, a default broker self-terminated after 1024 traps. Found because the K=200 benchmark crossed it on invocation four; fixed in v0.7; the budget is simply gone. (The v0.7 broker served one connection per request, serially; the v0.8 broker accepts concurrently under a bounded active-session budget with per-connection I/O timeouts, and a silent or hostile peer consumes one slot without head-of-line-blocking other cells, `process_boundary.rs::silent_peer_does_not_head_of_line_block_other_cells`, while a malformed or oversized frame produces a structured per-connection error instead of killing the daemon.) (2) The warm-capsule short-circuit existed only on a legacy test path, never on the production syscall path: 200 reads of the same document cost 200 traps. Fixed by the consumption-tracking work above; the S10 re-run asserts the fix on the wire (the same workload is now one trap, Section 6.1). Both are reported here because they are exactly the kind of defect only a real end-to-end pipeline surfaces.

Implementation obligations: what v0.8 discharged, and what remains. The v0.7 draft recorded five obligations; the v0.8 closure discharged most of them in the product, and we restate the residue rather than delete the list. *Discharged:* per-deployment key provisioning: `aster-init` generates the seal key, launch key, and DB URL as `0600` files read at boot (`ASTER_SEAL_KEY_FILE`, `ASTER_LAUNCH_KEY_FILE`), replacing the test-fixture derivation on the production path; egress-blocked cells: the production Compose profile runs cells rootless with `network_mode: none`, read-only root filesystems, all capabilities dropped, `no-new-privileges`, PID/CPU/memory/FD limits and bounded tmpfs, the database network attached to the broker alone, and kernel `SO_PEERCRED` UID admission on the socket (`docker/compose.production.yml`); mint-time launch identity: the one-use launch token of Section 3.2; and resource exhaustion: deployment policy bounds per-transaction reads/scans and session count/TTL (`crates/ipc/src/policy.rs`), wire frames are size-bounded with a structured `response_too_large` error instead of a daemon crash, module bundles are size-capped before reading, and runaway JavaScript dies by V8 heap limit, watchdog `terminate_execution`, and the supervisor's external timeout (`crates/v8cell/tests/resource_limits.rs`). *Remaining:* κ lifecycle beyond files (KMS/secret-store integration and rotation); per-field byte caps on capsule strings and documents (the shipped caps are count- and frame-level); epoch delivery to the *cell*, which still rides the supervisor's launch environment, now cross-checked at mint by the token's epoch binding, while the postgres-mode broker derives its own epoch from `acquire_lease` at boot and ignores `ASTER_LEASE_EPOCH` outright (memory mode keeps the env stand-in); and the operational cadence of retention-watermark advancement across committer failover: the safety half (clamp, pin, and the R0 repair at lease acquisition) is tested; the cadence half is an operator policy.

6 Evaluation

Numbers in this section are measured; where a number does not exist yet we say so and give the planned methodology instead. Nothing here is projected. One dating note applies throughout: both campaigns predate the v0.8 closure: the single-history store, deployment policy, launch tokens, and the hardened profile were absent from every measured build; the v0.8 path is not yet re-measured.

Setup. Two campaigns, same developer workstation (AMD Ryzen 7 5800H, 16 hardware threads, 31 GiB RAM, Arch Linux; a docker dev stack idles alongside; the load snapshot is in the committed logs). The **v0.6 campaign** (2026-07-16, pre-fix): docker images built from the repository, one-shot cell *containers* against a long-lived broker container, `postgres:16` fixture store. The **v0.7 campaign** (S10, 2026-07-16, harness `bench/run-v07.sh`, raw logs + consolidated report in `bench/results/v07/`): release binaries as bare host processes, `postgres:16.14` in a container with stock durability (`synchronous_commit=on`, `fsync=on`), fresh bench database per run (Convex-schema fixtures for the read plane, the `aster` schema for the write plane). Method for the read benches: JavaScript workloads T0 (no syscall), T1 (one `db.get`), TK (K=200 sequential gets), each run N=12 times as a fresh one-shot cell; spawn cost cancels in the subtractions and trap counts are sanity-asserted from the cell's own `"traps":N` output. Write benches: warmup discarded where a warmup pass exists (blind 100 / point-validation 30 / e2e 30; the window-validation and conflict-abort phases run un-warmed after hot phases; the campaign report carries the note), then ≥100–1,500 samples per point (stated per table) with median/p95 reported and full per-sample series committed.

6.1 EQ1: the read-path security tax

Table 4: v0.6 read-path baseline (p50 unless noted, N=12, K=200, dockerized, 2026-07-16).

Measurement	Value
Cold one-shot invocation, no reads (T0)	390 ms (min 351, p95 398)
Cold one-shot invocation, 1 read (T1)	390 ms (min 362, p95 400)
Cold one-shot invocation, 200 reads (TK)	458 ms (min 422, p95 490)
Marginal cost per trap, (TK − T1)/199	**0.34 ms**
First-trap cost (T1 − T0)	below measurement noise (< ±10 ms)
Implied serial broker throughput	~2,900 traps/s (single-threaded, one connection per trap)

That baseline measured an always-trap pipeline: 200 reads of the same id cost 200 traps over a constant 1-entry capsule, 0.34 ms each (0.3–0.5 ms across configurations): the number the abstract of the v0.6 draft carried. Two v0.7 changes altered what the same workloads mean, so S10 re-ran them (host processes instead of containers; N=12, K=200; `bench/results/v07/b1-read-path.log`):

Table 4b: v0.7 read-path re-run (p50, N=12, K=200, host processes, S10).

Measurement	Value
T0 (no reads)	5 ms (min 5, p95 6)
T1 (one read)	6 ms (min 6, p95 7)
TK, same key ×200	7 ms, **1 trap** (harness-asserted; was 200 traps)
TK, distinct keys ×200	191 ms, 200 traps (min 183, p95 208)
First-trap cost (T1 − T0)	~1 ms

Measurement	Value
Warm-read marginal ((TKsame − T1)/199)	≤0.005 ms/read (timer floor; B1's 1 ms granularity makes the derived marginals upper bounds)
Cold-trap marginal ((TKdist − T1)/199)	0.93 ms/trap (capsule grows 1→200)

What changed, stated plainly. (1) The warm-capsule fix (Section 5) removed the workload the 0.34 ms number measured: a read the capsule already holds no longer traps at all: 200 same-key reads now cost one trap plus ≈5 μs per warm read inside V8. (2) A cold trap now runs **two** store queries, not one: the value read plus the retention-floor guard against Convex's document vacuum (`min_document_snapshot_ts`), a review finding fixed between the campaigns; the price of refusing to mint sealed evidence from half-vacuumed history is one extra Postgres round trip per trap. The 0.93 ms cold-trap marginal is therefore two Postgres round trips plus a growing-capsule reseal; the apparatus share of it is 0.03–0.16 ms at these capsule sizes (EQ2 below measures exactly that split). The v0.6 conclusion survives in sharper form: **the cryptography and broker hop are not where a read's cost lives; the store round trips are.**

6.2 EQ2: reseal scaling

The v0.6 caveat is now a measured curve. The reseal re-encodes the whole capsule (O(capsule bytes) per trap), so S10 measured per-trap cost at *constant* capsule size n (grow to n distinct entries, then 300 samples re-hydrating an already-held key over the real UDS wire, memory store so no Postgres term; `bench/results/v07/b2-reseal-curve.log`):

Table 5: per-trap apparatus cost vs. capsule size (median of 300, S10).

n (entries)	wire request	per trap	cumulative climb to n
1	0.9 KB	0.035 ms	0.1 ms
10	1.3 KB	0.040 ms	0.5 ms
100	5.8 KB	0.096 ms	6.5 ms
200	10.6 KB	0.157 ms	19.2 ms
500	25.4 KB	0.354 ms	103.9 ms
1000	49.8 KB	0.697 ms	366.9 ms

The shape is what the design predicts and we state it plainly: **linear per trap in capsule size** (least-squares 0.030 ms + 0.66 μs per held entry, a slight upper bound: the harness's timed closure includes a client-side capsule clone a real cell doesn't pay, ≈13.5 μs per KB of capsule crossing the wire twice; the fit predicts 0.692 ms at n=1000 vs 0.697 measured) and therefore **quadratic cumulative** over a read-set built one trap at a time (Σ predicts 361 ms for the n=1000 climb; 367 ms measured). At n=1000 with these ~49-byte entries the whole apparatus (0.70 ms) still sits below what the store's own two snapshot queries cost a cold trap (~0.8 ms of EQ1's 0.93 ms marginal), so the quadratic is real but not yet the bottleneck at realistic read-set sizes; entries scale it by their byte size, and incremental or Merkle-ized sealing remains the known remedy past ~10^3 entries or fat documents, still deliberately unbuilt ahead of need.

6.3 EQ3: commit throughput and abort behavior

Measured in two layers (S10; `bench/results/v07/{b3-fence-isolated,b4-e2e-write}.log`): the fence in isolation, via direct `WritePlane::commit` in a fresh (tenant, deployment) namespace with a single

serial committer as the lease design mandates, and the full loop from inside a V8 cell. Per fence, the SQL round-trip count is fixed by construction: session-GUC stamp, BEGIN, lease `FOR UPDATE`, horizon read, retention pin `FOR UPDATE`, one INSERT per write, COMMIT; that is **7 round trips for a blind 1-write commit**. A point set adds one `ANY($n)` scan, windows add one DISTINCT-key scan over (s, h].

Table 6: the fence in isolation (median/p95, stock Postgres 16 in docker).

Case	median	p95	N
Blind 1-write commit	3.51 ms	3.87	1,500; **280 commits/s sustained serial**
+ point validation, p = 1 / 10 / 50 / 200	4.29 / 4.29 / 4.28 / 3.90 ms	≤4.8	200 each
+ window validation, w = 1 / 10 / 50, over a growing ~1.0k→1.6k-event (s, h]	4.59 / 5.81 / 6.61 ms	≤7.3	200 each
Conflict-abort (declared point, interposed write)	**1.75 ms**	1.98	100

The results are consistent with a durability-bound fence: the blind commit's 3.5 ms tracks the synchronous WAL flush at COMMIT (an inference, not an isolated attribution; no durability-off control was run), and an *abort is half the price of a commit*: same lease lock, same horizon, same conflict scan, no flush (the 1.75 ms conflict figure is decision latency, measured before the fence gained its awaited rollback). Point validation is one extra round trip whose cost is flat in p over the tested *empty* conflict suffix (an = `ANY` probe; the p=200 median landing below p=1 is run-to-run noise, ~0.4 ms; cost under a populated suffix is unmeasured). Window validation pays the DISTINCT-key scan over the (s, h] population (~1.1 ms at ~1k events) plus in-committer interval matching; the sweep's three w-phases share one growing log (~1.0k events at w=1, ~1.6k by w=50; each committed sample appends), so the +2 ms from w=1 to w=50 is an upper bound that conflates window count with population growth. 280 serial commits/s is the observed throughput of this implementation and configuration, consistent with an fsync-per-commit regime; we claim no ceiling. Cross-transaction group commit is *not* a drop-in lever here: the lease-row lock is held until COMMIT completes, so batching several logical transactions would require a modified fence.

Table 7: the full plumbing loop (read adapter and commit log are separate histories; Section 5), one transaction per iteration (N=500 after 30 warmup).

Leg	median	p95
Cell exec: InitialCapsule + fresh V8 isolate + JS (`1.0/get` trap ×1 + `1.0/insert`)	2.44 ms	2.78
Commit: `Commit` verb over UDS + session gate + seal verify + fence	4.03 ms	4.42
Total per transaction	**6.49 ms**	7.10

Sustained: 153 tx/s, serial, every transaction a real JS mutation in a fresh isolate whose write set crosses the commit fence (`Committed` asserted per sample; read store is the Postgres Convex adapter, fence is the

Postgres write plane; both sides real). The cross-check that matters: the commit leg (4.03 ms) lands close to the isolated fence with one point validation (4.29 ms). This campaign has no paired A/B control, so the supportable claim is that **no write-path apparatus overhead (UDS round trip, session end-of-life, seal verification, B-SUBSET declaration check) was isolated by this comparison**, not that it is zero, consistent with EQ2's 0.035 ms apparatus floor at small capsules; the policy and launch-security mechanisms contribute no cost here because they did not exist in the measured build; they shipped in v0.8, and their cost is unmeasured. Not yet measured, named so the reader can hold us to it: abort *rate* under contention (only the per-abort cost is measured), point validation under a populated suffix, window cost at a reset fixed suffix, multiwrite transactions, queued latency under concurrent callers, fresh-snapshot retry (which in v0.7 requires a broker relaunch), prewarm on/off sweeps, and the failover blip across a lease takeover.

6.4 EQ4: end-to-end invocation cost

The invocation floor is spawn plus V8 boot, and the two campaigns bracket it: ~390 ms as a docker one-shot container (Table 4), ~6 ms as a bare host process (Table 4b, T0); the capsule machinery contributes ~1 ms at one read in either case. The floor is deployment packaging, not protocol: a warm cell pool (reincarnation) remains the roadmap item, and warm *serving* latency will be reported when one exists, not projected. What is measured today: 200 distinct authenticated reads complete in 191 ms end-to-end as a host one-shot (Table 4b), and a full read-then-mutate transaction costs 6.5 ms warm-process (Table 7), for one-shot analytical offload and for serial mutation work respectively. We make no SLO claim from these two numbers, and memory pressure on the reactive backend is not measured; the offload argument is architectural (the cell plane holds no resident subscription state), not yet quantified.

6.5 EQ5: comparison baselines [PENDING: baseline campaign]

Two baselines are planned and remain unmeasured; S10 scoped them out (it measures Aster against itself; the comparison rig is a different fixture). (a) The same query/mutation as a trusted-executor Convex function in the upstream backend, quantifying what the isolation actually costs end-to-end. (b) A TEE-based deployment as a concept baseline: the argument of Section 7 is that a trusted broker makes the enclave unnecessary for *executor-to-database authority* under Aster's trust assumptions, not for confidentiality against a compromised host and not for side channels; the measurable form of that argument is one broker process versus an attestation pipeline. Neither number exists yet, and nothing above stands in for them.

7 Related work

Table 1: positioning by trust locus × mechanism × guarantee × cost.

System	Untrusted party	Mechanism	Guarantee	Cost
FoundationDB [16]	nobody (client in TCB)	declared conflict ranges + OCC resolver	serializability *if clients honest*	—
Basil [13]	clients + replicas	BFT quorums, 5f+1, commit certificates	Byzantine-tolerant serializability	replication ×5+

System	Untrusted party	Mechanism	Guarantee	Cost
Hyperledger Fabric [3]	peers/clients	endorser re-execution + signed rwsets, version validation	endorsement-policy integrity	re-execution ×endorsers
Fides/TFCommit [4]	storage + coordinator + clients (no online trusted party)	CoSi-signed hash-chained log + Merkle state; offline auditor	**detection** (auditability), post-hoc	audit infrastructure
TransEdge [12]	edge storage replicas	BFT-SMaRt per partition + Merkle proofs + f+1 quorum signatures to trusted clients	serializable reads vs untrusted storage	replication 3f+1
Ryoan / Opaque / EnclaveDB [2, 15, 9]	executor/operator hosts	SGX enclaves + attestation (Opaque: dataflow self-verification)	confidentiality/integrity via hardware	TEE + attestation pipeline
Cobra [14]	the database	offline SMT serializability checking by trusted clients	detection, post-hoc	audit compute
Aster	**the executor (arbitrary app code)**	**serve-time keyed-MAC capsules + OCC backward validation at the lease-holding broker**	**online prevention: strict serializability over declared, authenticated sets**	**one broker process**

FoundationDB is the mechanism anchor. Its resolver performs exactly the backward validation of T2: clients obtain a read version, read, and submit read/write conflict ranges; resolvers check submitted read ranges against recently modified ranges; snapshot reads are omitted from conflict sets [16]. The client, however, is in the TCB: an executor that under-declares commits what OCC should have aborted. Aster's exact delta is provenance-authenticity of each submitted observation (the executor cannot fabricate the resolver's evidence), while semantic completeness remains governed by the variant and rollback boundary (Section 4, T2 remark).

Fides / TFCommit is closest in spirit, and owns the no-BFT precedent. TFCommit commits serializable transactions across multiple *untrusted* storage servers without Byzantine replication, using a hash-chained, collectively signed (CoSi) tamper-evident log over Merkle-authenticated datastores [4]. Fides and Aster share the goal of transactional integrity on untrusted infrastructure while avoiding BFT state-machine replication, but they invert the trust model and, with it, the guarantee. Fides trusts no online party (storage, coordinator, and clients may all be Byzantine) and therefore offers *auditability*: incorrect executions are not prevented but are made irrefutable and detected after the fact by an external, offline auditor reconstructing a serialization graph from the signed log. Aster places a single trusted, lease-holding broker in the online data path and treats only the executors as Byzantine, allowing it to *prevent* rather than detect: every served read is sealed at serve time with a keyed MAC binding it to (cell, lease epoch, snapshot), and the broker validates the authenticated

read-set through backward validation before any commit is admitted. The concentration of trust also changes the cryptography: Fides needs publicly verifiable collective signatures precisely because it has no trusted verifier; Aster's symmetric MAC is sound only because the broker both seals and checks. Fides authenticates the *log* for post-hoc dispute resolution; Aster authenticates *per-read read-sets* to make OCC sound against Byzantine executors online: a mechanism, and a strict-serializability safety guarantee, that a detection-based design does not provide.

Basil shows Byzantine *clients* plus serializability is achievable with BFT machinery: 5f+1 replicas per shard and commit certificates yield Byzantine-tolerant serializability [13]. Aster targets the same robustness against the application principal at a different cost point (one trusted broker process), because its threat model concedes a trusted storage authority. The goals overlap; the cost models do not, and Aster is *not* a one-node replacement for Byzantine storage replication.

Hyperledger Fabric validates signed read/write sets at commit time without re-executing there, but those rwsets originate from *endorsers who executed the chaincode* [3]; integrity is rooted in re-execution by a quorum of trusted-enough peers. Aster removes the endorsers: nothing re-executes the tenant computation; the trusted broker signs *data flow* at serve time, and the write set is treated as adversary-chosen but policy-gated.

TransEdge is closest in vocabulary (Byzantine transaction processing, authenticated reads, no trusted hardware), with the trust boundary in the opposite location: its untrusted parties are the edge *storage* replicas, defended with classical BFT state-machine replication (3f+1 nodes per partition under BFT-SMaRt), and reads are made verifiable to a *trusted client* via Merkle proofs carrying f+1 replica signatures with dependency-tracking for single-round cross-partition snapshots [12]. Aster inverts the locus: a single trusted broker owns storage and the lease; the *executor* is Byzantine; each served read is sealed with a single-issuer symmetric MAC so the broker can backward-validate the executor's self-certifying read-set at commit. Untrusted storage under replication versus untrusted executor under a trusted broker: complementary threat models whose read-authentication primitives (quorum signatures over Merkle roots vs. single-issuer MACs) are not interchangeable.

The TEE line (Ryoan [2], Opaque [15], EnclaveDB [9], VC3 [11]) confines untrusted or exposed execution with enclaves and attestation. Aster's position is that for *executor isolation against a database*, the enclave is unnecessary when a trusted broker authenticates the data flow: executor trustworthiness becomes irrelevant to isolation, which is the property T3 states. The trade is explicit: Aster spends a trusted online process where the TEE line spends attestation hardware, and Aster protects database authority, not the confidentiality of the executor's own computation.

Cobra validates serializability of a black-box *database* offline on behalf of trusted clients [14], the inverse trust direction, sharing the "don't verify the computation" spirit but as post-hoc detection. **Object-capability systems** (CapTP and the ocap tradition [6]) supply unforgeable references, and **IFC databases** (IFDB [10], Qapla [5]) confine queries by policy; neither wires unforgeable *read provenance* into an OCC serializability gate at commit time, which is precisely the composition here. Finally, a name-collision preempt: Berkeley's **DataCapsule /** Global Data Plane [7] names durable signed append-only logs: data at rest, unrelated to Aster's transient per-invocation read seals.

Against this map, Aster's contribution reduces to a specific, checkable composition: (i) serve-time MACs as the *online admission gate* of OCC validation, (ii) the *executor*, not storage, as the untrusted locus, and (iii) *prevention* at a trusted commit gate rather than post-hoc detection, with (iv) the no-TEE/no-BFT cost profile arriving as a consequence of the trusted-broker model rather than a claim.

8 Discussion, limitations, open problems

The weakest joint is A6, and we name it. Range soundness assumes *complete conflict projection*: every mutation that can change a point read or a key-interval scan surfaces as a write event on the corresponding primary/index key, visible to validation. A missing index-entry mutation, an off-by-one endpoint, or a GC/interposition race defeats T2 **without breaking any cryptography**. This is where a reviewer should attack, and where our own effort goes: index-space writes must be derived by the trusted committer (never the cell), the fence/GC/failover interleavings are model-checked in TLA+, and integrating with Convex's real index maintenance requires an audit of that code path before the write plane touches a live deployment. Until then A6 is an assumption we implement against our own log, not a verified property of an integrated system.

Side channels are excluded, not solved. "Nothing to exfiltrate" holds for *data the cell was never granted*; it does not extend to covert channels. Timing, cache, and scheduler channels are outside the model (A12), and the deployment obligation is real: cells must run with egress blocked, or exfiltration of authorized data is trivial. The conflict bit of T1b is a named, authorized leakage (one bit about whether an authorized window changed), and a data-dependent read policy leaks its allow/deny outcomes by construction.

Staleness is the headline's fine print. Strict serializability is for *mutations*; a read-only invocation sees one consistent, possibly stale snapshot bounded by the retention rule. FoundationDB carries the same caveat for snapshot reads [16]; we repeat it every time the headline appears because omitting it would oversell the guarantee.

The proof is conditional and the system is not verified. The theorem is a specification with an explicit assumptions ledger (Table 3); the implementation discharges specific obligations with specific tests (Section 5), and the fence's abstract design is model-checked in TLA+ with its scope stated in Section 5 (serialization assumed; mutants, including the no-atomic-fence one, required to fail), an investment that already caught a real commit-admission liveness bug before any deployment; no end-to-end code-level verification is claimed. The reality-ledger discipline of the technical report, separating "implemented and tested today" from "proposed and covered conditionally", is carried into this paper's CLAIMS ledger.

Operational limits. Policy revocation across scaled read brokers propagates with bounded skew (F3). Long invocations lose commit liveness past the retention window and must retry (Lemma R). Exactly-once semantics require an application-level idempotency key persisted in the append transaction; capsule freshness is not deduplication (attack obligation 13).

Prototype seams the theorem does not hide. Four prototype seams are stated rather than papered over, each with its current (v0.8) status. (i) *Boot-pinned snapshots:* brokerd pins one snapshot and one lease epoch at launch, so a `Conflict/RetentionViolated` outcome's "retry from a fresh snapshot" contract is dischargeable only by relaunching the broker. The *history* half of this seam is closed: since v0.8 the Postgres-mode read plane serves the same `aster.log` the fence appends to, so a relaunched broker (and any subsequently launched cell) observes committed writes (`crates/ipc/tests/authoritative_postgres.rs`); the boot-pinning itself, live snapshot refresh inside one broker process, remains open. (ii) *Document-id identity and order:* the store accepts two spellings of the same row (Convex IDv6, plus a raw wire form), while every layer above (capsule keys, consumption ledger, write set, conflict scan) keys by the string; mixed spellings would evade read-your-own-writes and pairwise conflict detection, and dismissing that as a native-caller concern would contradict the threat model, because a Byzantine cell *is* a hand-rolled native caller. v0.7 therefore rejects the raw wire spelling for point documents at the broker seam (`noncanonical_document_id`), making IDv6 the only spelling a cell can use, and pins the scan order bytewise (`COLLATE "C"` in SQL, matching Rust's `str` order, regression-tested); full canonicalization inside the store adapter remains named work. (iii) *Policy:* since v0.8 the deployment policy is an explicit artifact (prefix-grained `P.read`/`P.write`/scan/module allowlists,

per-table insert gates, and transaction/session limits, wildcard only when explicit and deny-all in the generated skeleton; `crates/ipc/src/policy.rs`), so T1b's confinement statement is now as fine as the deployment's prefix policy. What remains unimplemented: policy versioning and revocation linearization (A10's versioning clause), and any per-principal authority below deployment scope: every session in a deployment shares one policy envelope. (iv) *Protocol leakage is broader than the conflict bit:* the wire outcomes reveal the exact conflicting key and the returned epoch, horizon, retention-floor, and commit-timestamp values: explicit protocol leakages, not side channels, and named as such in T1b's transcript; a hardened wire would return opaque conflict/retry classes. Module bundles are likewise broker-provided launch inputs outside the capsule theorem (Section 3).

Open problems. (1) *Reactive subscriptions over capsules:* invalidation without a resident write log in the cell plane; the promising shape is a broker-side approximate matcher with one-sided error: false positives cost only re-execution; false negatives are forbidden. The range-certificate language is a natural matcher input, but retention, compaction, and epoch handoff need their own proofs. (2) *Learned prewarming ("dream capsules"):* Variant B makes unused predictions free of conflict cost, but a learned selector must be policy-checked per item, must not encode another tenant's secrets in its choices, and capsule size itself may become a side channel or DoS vector. (3) *Zero-knowledge compliance transcripts:* prove capability/protocol compliance without revealing documents, useful as an audit layer, never to be marketed as proof of JavaScript correctness. (4) *Actual-dependency completeness without trusting V8*, the clean open question the repaired theorem exposes: can a system keep prewarm and stateless read serving while proving that every value causally consumed by a Byzantine executor appears in R? A MAC cannot attest to *use*; every known direction (one-time finalization state, a trusted monitor on cache hits, hardware attestation, proof-carrying execution) spends a resource this design deliberately avoids.

9 Conclusion

Aster makes every submitted OCC observation provenance-authentic without trusting or re-executing the application executor; strict serializability of committed mutations then follows from classical backward validation (snapshot reads stay serializable but possibly stale), while any omitted dependency is demoted to an authorized blind write rather than a cross-authority isolation failure. The cut that pays off is refusing to make the executor trustworthy (no enclave, no replication, no re-execution) and making its *data* unforgeable instead. Under that cut, in a deployment that enforces A12's egress, filesystem, process, and socket isolation and provisions the MAC key outside the code (a profile the v0.8 artifact ships as a hardened Compose deployment: rootless, network-less cells, kernel peer-credential admission, file-provisioned keys, not independently audited), a V8 compromise adds no *database authority* beyond the broker protocol: the attacker holds a socket to a broker that authenticates every read it ever served and a commit gate that validates authority, not intentions, and a compromised executor collapses to a malicious-but-authorized application. That confinement is database-authority confinement, not data-loss prevention: keeping authorized data private still requires the egress isolation. The apparatus costs ~0.04 ms per authenticated read at the measured read-set sizes (linear in capsule size, free when the capsule already holds the key), and a fenced mutation from inside an untrusted cell lands in ~6.5 ms in a warm-process benchmark against Aster's own log; the write path is built on its own log and fence, proved and measured against real Postgres (since v0.8 the standalone plane reads from that same authoritative history), with integration into a live Convex deployment's committer named as the honest remaining work.

Appendix pointer

The full model, repairs, proofs, thirteen-attack appendix, assumptions ledger, and variant verdict are in the companion technical report [1]: *The Capsule Transaction Theorem — a repaired proof for Aster v0.7* (27 pp., July 2026), as amended by its *Seal-v3 and Session Protocol Addendum* (July 2026), which closes the previously declared version skew (re-referee F9) by proving the shipped v3 protocol exactly: the session frame enters the outer-message injectivity lemma (Lemma 3.2-v3), the T1a reduction becomes one-branch EUF-CMA with the unkeyed-collision assumption (A3) retired, T1a is stated in the honest bearer-session scope, the consume-first session lifecycle enters CE 2.1, T1b names the exact whole-protocol transcript leakage, retention gains the standalone well-formedness invariant R0, and the implemented-vs-proposed ledger is re-stamped to the current tree. Section 4 of this paper is a faithful condensation of the repaired abstract protocol. The companion report, as amended by that addendum, governs the formal protocol claims: it proves the shipped v3 direct-MAC/bearer-session protocol and the Aster-owned single-history fence, and it does not erase the separately disclosed F1 two-plane limitation.

References

[1] Ian Lucas Bée. The Capsule Transaction Theorem — a repaired proof for Aster v0.7, as amended by its Seal-v3 and Session Protocol Addendum. Companion technical report, 27 pp. + addendum, https://github.com/Iann29/aster/tree/main/paper/sources, 2026.

[2] Tyler Hunt, Zhiting Zhu, Yuanzhong Xu, Simon Peter, and Emmett Witchel. Ryoan: A distributed sandbox for untrusted computation on secret data. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*. USENIX Association, 2016.

[3] Hyperledger Fabric. Architecture explained: Endorsing peer simulation and read/write sets. Hyperledger Fabric documentation, https://hyperledger-fabric.readthedocs.io/en/latest/arch-deep-dive.html. Accessed July 2026.

[4] Sujaya Maiyya, Danny Hyun Bum Cho, Divyakant Agrawal, and Amr El Abbadi. Fides: Managing data on untrusted infrastructure. In *Proceedings of the 40th IEEE International Conference on Distributed Computing Systems (ICDCS 2020)*. IEEE, 2020. arXiv:2001.06933.

[5] Aastha Mehta, Eslam Elnikety, Katura Harvey, Deepak Garg, and Peter Druschel. Qapla: Policy compliance for database-backed systems. In *Proceedings of the 26th USENIX Security Symposium*. USENIX Association, 2017.

[6] Mark Samuel Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006. Defines the CapTP distributed capability protocol.

[7] Nitesh Mor, Richard Pratt, Eric Allman, Ken Lutz, and John Kubiatowicz. Global Data Plane: A federated vision for secure data in edge computing. In *Proceedings of the 39th IEEE International Conference on Distributed Computing Systems (ICDCS 2019)*. IEEE, 2019.

[8] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: one function, fast everywhere. Specification and design paper, https://github.com/BLAKE3-team/BLAKE3-specs, 2020.

[9] Christian Priebe, Kapil Vaswani, and Manuel Costa. EnclaveDB: A secure database using SGX. In *Proceedings of the 2018 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2018.

[10] David Schultz and Barbara Liskov. IFDB: Decentralized information flow control for databases. In *Proceedings of the 8th ACM European Conference on Computer Systems (EuroSys '13)*. ACM, 2013.

[11] Felix Schuster, Manuel Costa, Cédric Fournet, Christos Gkantsidis, Marcus Peinado, Gloria Mainar-Ruiz, and Mark Russinovich. VC3: Trustworthy data analytics in the cloud using SGX. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2015.

[12] Abhishek A. Singh, Aasim Khan, Sharad Mehrotra, and Faisal Nawab. TransEdge: Supporting efficient read queries across untrusted edge nodes. In *Proceedings of the 26th International Conference on Extending Database Technology (EDBT 2023)*, pages 684–696, 2023. arXiv:2302.08019.

[13] Florian Suri-Payer, Matthew Burke, Zheng Wang, Yunhao Zhang, Lorenzo Alvisi, and Natacha Crooks. Basil: Breaking up BFT with ACID (transactions). In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*. ACM, 2021.

[14] Cheng Tan, Changgeng Zhao, Shuai Mu, and Michael Walfish. Cobra: Making transactional key-value stores verifiably serializable. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*. USENIX Association, 2020.

[15] Wenting Zheng, Ankur Dave, Jethro G. Beekman, Raluca Ada Popa, Joseph E. Gonzalez, and Ion Stoica. Opaque: An oblivious and encrypted distributed analytics platform. In *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI '17)*. USENIX Association, 2017.

[16] Jingyu Zhou, Meng Xu, Alexander Shraer, Balamurugan Namasivayam, Alex Miller, Evan Tschannen, Steve Atherton, Andrew J. Beamon, Rusty Sears, John Leach, Dave Rosenthal, Xin Dong, Will Wilson, Ben Collins, David Scherer, Alec Grieser, Young Liu, Alvin Moore, Bhaskar Muppana, Xiaoge Su, and Vishesh Yadav. FoundationDB: A distributed unbundled transactional key value store. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*, pages 2653–2666. ACM, 2021.